Assignment 2:

Name: Leon Lobo
Admission No: U24AI043

Q1. The following figure shows the start state and goal state for the 8-puzzle problem. Find the number of states explored before reaching the goal state using the BFS algorithm.

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Code:

Lab 2 > A2_Q1.py > bfs

```python
def bfs(puzzle,goal,moves):
    queue=[puzzle]
    front=0
    visited={puzzle}
    explored=0

    while front<len(queue):
        state=queue[front]
        front+=1

        if state==goal:
            return explored

        explored+=1
        blank=state.index(0)

        for move in moves[blank]:
            next_state=list(state)
            next_state[blank],next_state[move]=next_state[move],next_state[blank]
            new_state=tuple(next_state)

            if new_state not in visited:
                visited.add(new_state)
                queue.append(new_state)



def main():
    puzzle=(7,2,4,
            5,0,6,
            8,3,1)

    goal=(0,1,2,
          3,4,5,
          6,7,8)

    # Possible moves of blank tile (by index)
    moves={{
            0: [1,3],
            1: [0,2,4],
            2: [1,5],
            3: [0,4,6],
            4: [1,3,5,7],
            5: [2,4,8],
            6: [3,7],
```

Lab 2 > A2_Q1.py > main

```python
def main():
            8: [5,7]
        },
        {
            0: [1,3],
            1: [2,4,0],
            2: [5,1],
            3: [4,6,0],
            4: [5,7,3,1],
            5: [8,4,2],
            6: [7,3],
            7: [8,6,4],
            8: [7,5]
        },
        {
            0: [3,1],
            1: [4,0,2],
            2: [5,1],
            3: [6,0,4],
            4: [7,3,1,5],
            5: [8,4,2],
            6: [3,7],
            7: [6,4,8],
            8: [7,5]
        },
        {
            0: [1,3],
            1: [0,2,4],
            2: [1,5],
            3: [0,4,6],
            4: [3,1,5,7],
            5: [4,2,8],
            6: [3,7],
            7: [6,4,8],
            8: [7,5]
        }]

    direction=int(input("What direction should be prioritised? 0:Up 1:Right 2:Down 3:Left \n Number:"))

    no_of_states=bfs(puzzle,goal,moves[direction])
    print("Number of states explored: ",no_of_states)


if __name__=='__main__':
    main()
```

Output:

```
PS C:\Education\SVNIT\Coding\AI> python -u "c:\Education\SVNIT\Coding\AI\Lab 2\A2_Q1.py"
What direction should be prioritised? 0:Up 1:Right 2:Down 3:Left
 Number:0
Number of states explored:  169740
PS C:\Education\SVNIT\Coding\AI> python -u "c:\Education\SVNIT\Coding\AI\Lab 2\A2_Q1.py"
What direction should be prioritised? 0:Up 1:Right 2:Down 3:Left
 Number:1
Number of states explored:  172833
PS C:\Education\SVNIT\Coding\AI> python -u "c:\Education\SVNIT\Coding\AI\Lab 2\A2_Q1.py"
What direction should be prioritised? 0:Up 1:Right 2:Down 3:Left
 Number:2
Number of states explored:  170323
PS C:\Education\SVNIT\Coding\AI> python -u "c:\Education\SVNIT\Coding\AI\Lab 2\A2_Q1.py"
What direction should be prioritised? 0:Up 1:Right 2:Down 3:Left
 Number:3
Number of states explored:  165018
```

Q2 Find the number of states explored for the DFS algorithm. Give comparison of BFS and DFS in terms of cost if the path cost for the state is given as depth of the search tree at which the state is occurring.

Code:

Lab 2 > A2_Q2.py > dfs

```python
def dfs(start, goal, moves):
    stack = [start]
    visited = {start}
    explored = 0

    while stack:
        state = stack.pop()

        if state == goal:
            return explored

        explored += 1
        blank = state.index(0)

        for move in moves[blank]:
            next_state = list(state)
            next_state[blank], next_state[move] = next_state[move], next_state[blank]
            new_state = tuple(next_state)

            if new_state not in visited:
                visited.add(new_state)
                stack.append(new_state)



def main():
    puzzle=(7,2,4,
            5,0,6,
            8,3,1)

    goal=(0,1,2,
          3,4,5,
          6,7,8)

    # Possible moves of blank tile (by index)
    moves=[{
            0: [1,3],
            1: [0,2,4],
            2: [1,5],
            3: [0,4,6],
            4: [1,3,5,7],
            5: [2,4,8],
            6: [3,7],
            7: [4,6,8],
            8: [5,7]
```

Lab 2 > A2_Q2.py > dfs

```python
def main():
            7: [4,6,8],
            8: [5,7]
        },
        {
            0: [1,3],
            1: [2,4,0],
            2: [5,1],
            3: [4,6,0],
            4: [5,7,3,1],
            5: [8,4,2],
            6: [7,3],
            7: [8,6,4],
            8: [7,5]
        },
        {
            0: [3,1],
            1: [4,0,2],
            2: [5,1],
            3: [6,0,4],
            4: [7,3,1,5],
            5: [8,4,2],
            6: [3,7],
            7: [6,4,8],
            8: [7,5]
        },
        {
            0: [1,3],
            1: [0,2,4],
            2: [1,5],
            3: [0,4,6],
            4: [3,1,5,7],
            5: [4,2,8],
            6: [3,7],
            7: [6,4,8],
            8: [7,5]
        }]

    direction=int(input("What direction should be prioritised? 0:Left 1:Up 2:Right 3:Down \n Number:"))
    no_of_states=dfs(puzzle,goal,moves[direction])
    print("Number of states explored: ",no_of_states)


if __name__=='__main__':
    main()
```

Output:

```
PS C:\Education\SVNIT\Coding\AI> python -u "c:\Education\SVNIT\Coding\AI\Lab 2\A2_Q2.py"
What direction should be prioritised? 0:Left 1:Up 2:Right 3:Down
 Number:0
Number of states explored:  125880
PS C:\Education\SVNIT\Coding\AI> python -u "c:\Education\SVNIT\Coding\AI\Lab 2\A2_Q2.py"
What direction should be prioritised? 0:Left 1:Up 2:Right 3:Down
 Number:1
Number of states explored:  72229
PS C:\Education\SVNIT\Coding\AI> python -u "c:\Education\SVNIT\Coding\AI\Lab 2\A2_Q2.py"
What direction should be prioritised? 0:Left 1:Up 2:Right 3:Down
 Number:2
Number of states explored:  135294
PS C:\Education\SVNIT\Coding\AI> python -u "c:\Education\SVNIT\Coding\AI\Lab 2\A2_Q2.py"
What direction should be prioritised? 0:Left 1:Up 2:Right 3:Down
 Number:3
Number of states explored:  114255
```

Answer: In terms of cost, DFS would be higher even though it’s exploring fewer total states. This is due to the inherent nature of DFS as it explores one path(state) to the fullest and goes as deep as it can thus having explored deeper in the tree. BFS on the other hand explores many states at once but all at the same tree level leading to more states explored but at a cheaper cost.